

FreeRTOS基础（九）：FreeRTOS的列表和列表项

原创 VIP免费 已于 2024-07-19 07:55:49 修改 · 3k 阅读 ·  43 ·  52

今天我们将探讨FreeRTOS 中的一个核心概念——列表（List）和列表项（List Item）。在实时操作系统（RTOS）中，任务的管理和调度是至为重要的一环，而FreeRTOS使用列表来实现这一功能。列表可以说是FreeRTOS中的一种数据结构，通过列表，我们可以高效地组织、排序和管理任务、定时器、中断等系统资源。具体来说，列表项是组成列表的基本单位，每个列表项都包含指向具体任务或事件的指针，以及用于排序的数值。希望通过今天的学习，我们能够掌握利用列表和列表项来实现任务调度的原理，并能够独立看懂源码。

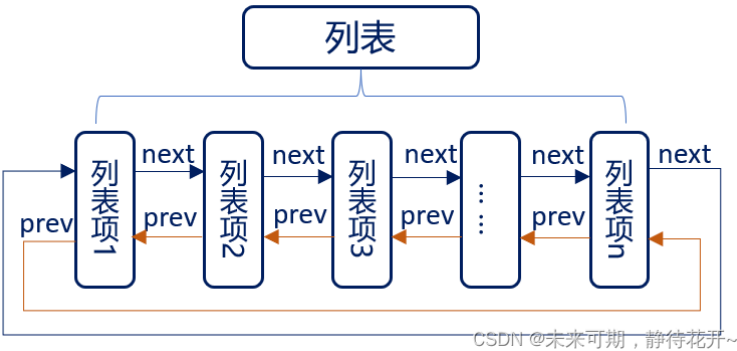
目录

- 一、列表和列表项简介
 - 1.1 列表和列表项的概念
 - 1.2 列表结构体
 - 1.3 列表项结构体
 - 1.4 迷你列表项结构体
 - 1.5 列表和列表项的关系
- 二、列表相关API函数介绍
 - 2.1 初始化列表函数vListInitialise ()
 - 2.2 初始化列表项函数vListInitialiseItem()
 - 2.3 列表项升序插入函数vListInsert()
 - 2.3.1 源码分析
 - 2.3.2 举例理解
 - 2.4 列表项尾插函数vListInsertEnd()
 - 2.4.1 源码分析
 - 2.4.2 举例理解
 - 2.5 列表项删除函数uxListRemove()
- 三、列表项的插入和删除实验
 - 3.1 创建任务及实现任务函数
 - 3.2 主函数调用入口函数，操作系统开始进行任务的切换和调度
 - 3.3 实验结果

一、列表和列表项简介

1.1 列表和列表项的概念

列表是 FreeRTOS 中的一个数据结构，概念上和链表有点类似，列表被用来跟踪 FreeRTOS中的任务。任务处在不同的状态下，就挂载在不同的列表（比如就绪列表、阻塞列表、挂起列表）。而列表项就是存放在列表中的项目，一个列表项就关联着一个任务，可以说：列表代表处于某种状态的任务，列表项就代表处于当前状态下的某一个任务。如下图所示：



举个例子：



列表相当于链表，列表项相当于节点，**FreeRTOS 中的列表是一个双向循环链表**，每个列表项有前驱结点指针prev，同时又有后继结点指针next，向循环链表的增删改查非常方便，动态改变，节省**内存**！列表的特点：列表项间的地址非连续的，是人为的连接到一起的。列表项的数目是由后期决定的，随时可以改变，而数组的特点：数组成员地址是连续的，数组在最初确定了成员数量后期无法改变。

在操作系统FreeRTOS中，任务的数量是不确定的，随时发生变化，并且任务状态是会发生变更的，所以非常适用列表(链表)这种数据结构。

1.2 列表结构体

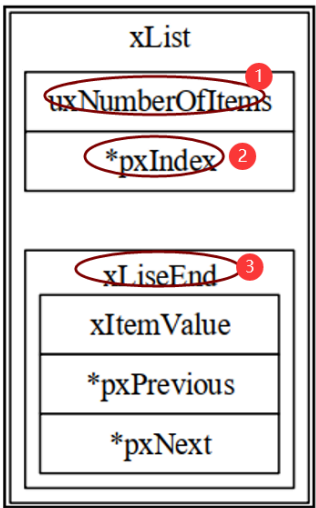
有关于列表的东西均在文件 list.c 和 list.h 中，首先我们先看下在list.h中的，列表相关结构体：

```
typedef struct xLIST
{
    listFIRST_LIST_INTEGRITY_CHECK_VALUE /* 校验值 */
    volatile UBaseType_t uxNumberOfItems; /* 列表中的列表项数量 */
    ListItem_t * configLIST_VOLATILE pxIndex /* 用于遍历列表项的指针 */
    MiniListItem_t xListEnd /* 末尾列表项 */
    listSECOND_LIST_INTEGRITY_CHECK_VALUE /* 校验值 */
} List_t;
```

CSDN @未来可期，静待花开~

1. 在该结构体中，包含了两个宏（第一个和最后一个），这两个宏是确定的已知常量，FreeRTOS通过检查这两个常量的值，来判断列表的运行过程中，是否遭到破坏，该功能一般用于调试，默认是不开启的；
2. 成员uxNumberOfItems，用于记录列表中列表项的个数（不包含末尾列表项，即最后一个列表项 xListEnd）；
3. 成员 pxIndex 用于指向列表中的某个列表项，一般用于遍历列表中的所有列表项（其实就是列表项指针）；
4. 成员变量 xListEnd 是一个迷你列表项，排在最末尾；

列表结构示意图



CSDN @未来可期，静待花开~

1.3 列表项结构体

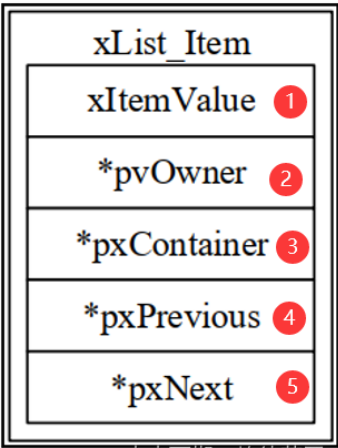
列表项是列表中**用于存放数据**的地方（**每一个列表项关联着一个任务，在列表项里面有一个成员变量，用来存放任务控制块，描述这个任务的信息**），在 list.h 文件中，有列表项的相关结构体定义：

```
struct xLIST_ITEM
{
    listFIRST_LIST_ITEM_INTEGRITY_CHECK_VALUE /* 用于检测列表项的数据完整性 */
    configLIST_VOLATILE TickType_t xItemValue /* 列表项的值 */
    struct xLIST_ITEM * configLIST_VOLATILE pxNext /* 下一个列表项 */
    struct xLIST_ITEM * configLIST_VOLATILE pxPrevious /* 上一个列表项 */
    void * pvOwner /* 列表项的拥有者 */
    struct xLIST * configLIST_VOLATILE pxContainer; /* 列表项所在列表 */
    listSECOND_LIST_ITEM_INTEGRITY_CHECK_VALUE /* 用于检测列表项的数据完整性 */
};
typedef struct xLIST_ITEM ListItem_t; /*类型重命名*/
```

CSDN @未来可期~

- 1、成员变量 xItemValue 为列表项的值，这个值多用于按升序对列表中的列表项进行排序，插入列表项的时候可根据此值来确定插入的位置。比如时阻塞的任务（对应一个列表项），它有一个延时时间，那么就根据延时时间在列表中进行排列，这样就可以实现优先解除时间短的任务。
- 2、成员变量 pxNext 和 pxPrevious 分别用于指向列表中列表项的下一个列表项和上一个列表项，其实就是双向循环链表的前驱结点指针和后继指针；
- 3、成员变量 pxOwner 用于指向包含列表项的对象（通常是任务控制块，存储该任务的相关属性信息）
- 4、成员变量 pxContainer 用于指向列表项所在的列表（当前列表项在哪种列表中，就绪？阻塞？挂起？）。即指向归属的列表！
通过成员3和成员4就可以确定该列表项属于哪个任务（任务控制块），并且在哪一种状态下。例如：第3个成员变量为任务1的控制块，第4个量为就绪列表，那么就可以确定任务1处在就绪状态下！

列表项结构示意图



CSDN @未来可期，静待花开~

1.4 迷你列表项结构体

迷你列表项也是列表项，但迷你列表项**仅用于标记列表的末尾和挂载其他插入列表中的列表项**，挂载如何理解：列表初始化的时候，只有一个项，当插入新的列表项时，就把新的列表项对应的两只手（前驱指针和后继指针）牵上迷你列表项即可！迷你列表项没有存储数据，不会存储实际的信息，因此不需要成员变量 `pxOwner` 和 `pxContainer`，以节省内存开销！

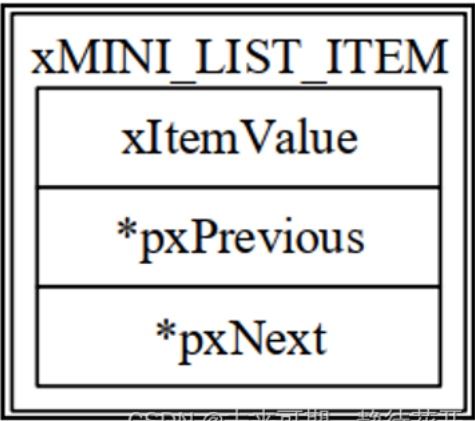
```
struct xMINI_LIST_ITEM
{
    listFIRST_LIST_ITEM_INTEGRITY_CHECK_VALUE
    configLIST_VOLATILE TickType_t xItemValue;
    struct xLIST_ITEM * configLIST_VOLATILE pxNext;
    struct xLIST_ITEM * configLIST_VOLATILE pxPrevious;
};
typedef struct xMINI_LIST_ITEM MiniListItem_t;
```

/* 用于检测数据完整性 */
/* 列表项的值 */
/* 上一个列表项 */
/* 下一个列表项 */

CSDN @未来可期，静待花开~

- 1、成员变量 `xItemValue` 为列表项的值，这个值多用于按升序对列表中的列表项进行排序
- 2、成员变量 `pxNext` 和 `pxPrevious` 分别用于指向列表中列表项的下一个列表项和上一个列表项

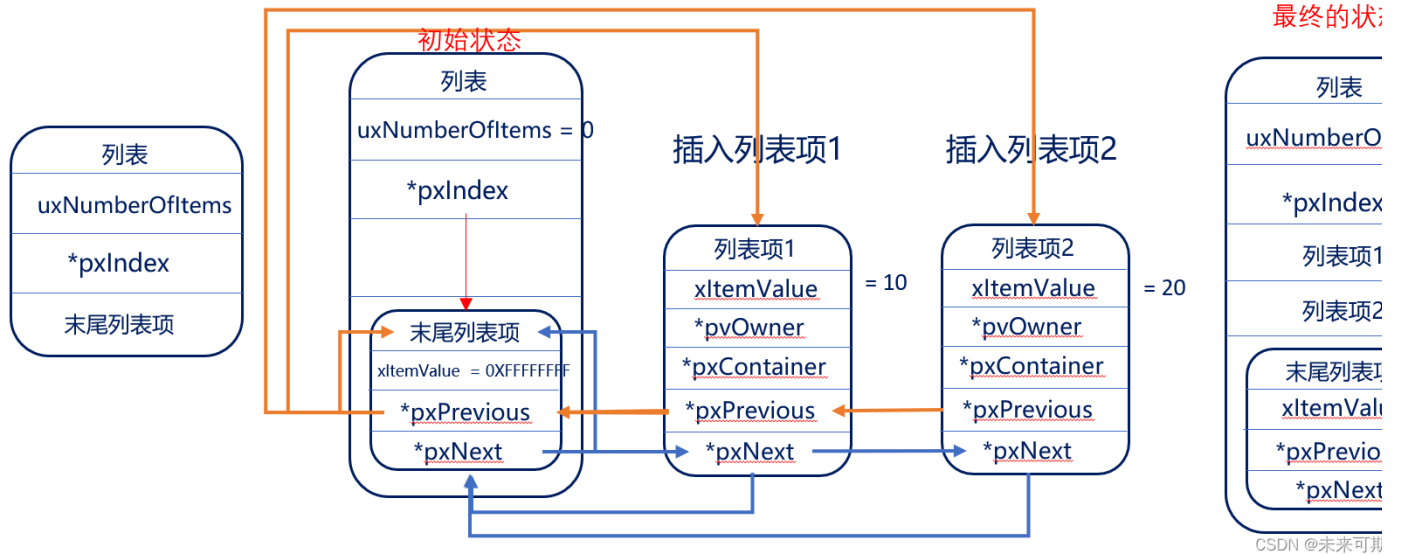
迷你列表项结构示意图



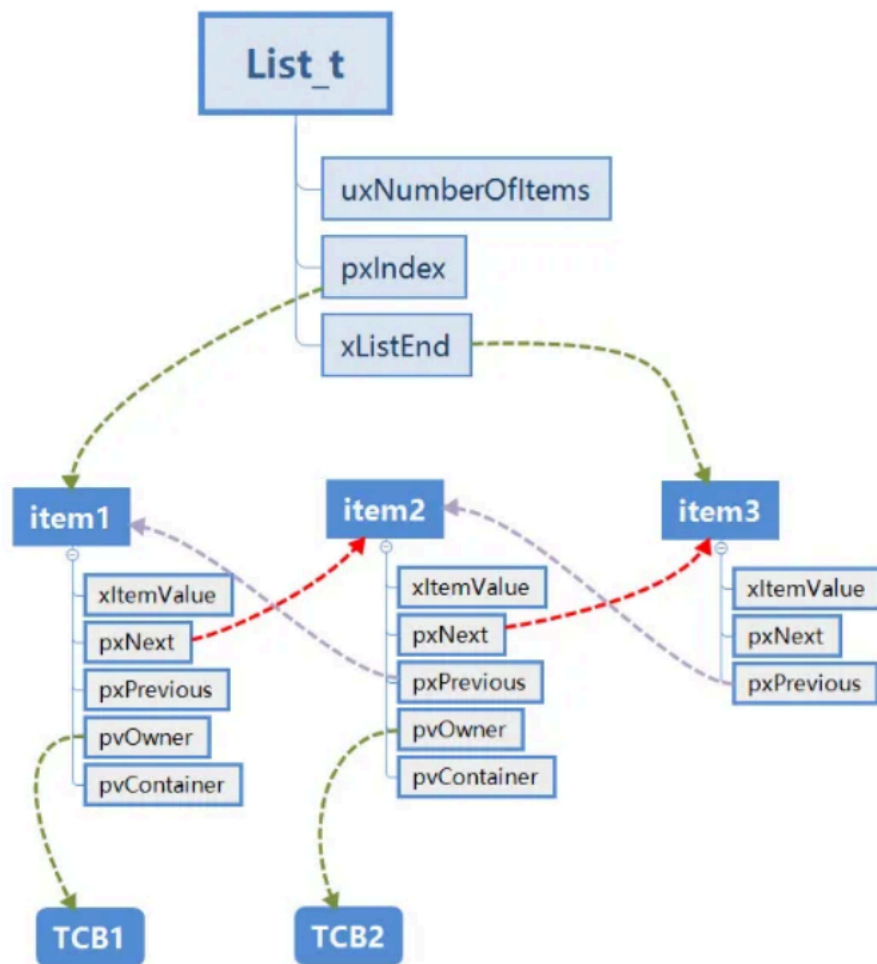
CSDN @未来可期，静待花开~

1.5 列表和列表项的关系

列表初始状态，以及即将插入的两个列表项如下：



宏观上看如下图：



CSDN @未来可期，静待花开~

二、列表相关API函数介绍

常见的列表的相关API函数如下，其实从数据结构上来讲：就是双向循环链表的初始化、增删操作。

函数	描述
vListInitialise()	初始化列表（初始化就是为结构体成员赋初值
vListInitialisemtem()	初始化列表项（初始化就是为结构体成员赋初
vListInsertEnd()	列表末尾插入列表项（尾插）
vListInsert()	列表插入列表项（按照升序插入）
uxListRemove()	列表移除列表项（删除列表项）

CSDN @未来可期，静

2.1 初始化列表函数vListInitialise ()

初始化列表就是为列表的结构体成员赋初值，此时列表只有一个末尾列表项。

cpp

AI写代码

```
1 函数原型
2 void vListInitialise( List_t * const pxList );
```

形参	描述
pxList	待初始化列表

```
void vListInitialise(List t * const pxList)
{
    /* 初始化时，列表中只有末尾列表项 xListEnd，因此 pxIndex 指向 xListEnd */
    pxList->pxIndex = ( ListItem_t * ) &( pxList->xListEnd );

    /* xListEnd 的值初始化为最大值0xffff ffff，用于列表项升序排序时，排在最后 */
    pxList->xListEnd.xItemValue = portMAX_DELAY;

    /* 初始化时，列表中只有 xListEnd，因此上一个和下一个列表项都为 xListEnd 本身 */
    pxList->xListEnd.pxNext = ( ListItem_t * ) &( pxList->xListEnd );
    pxList->xListEnd.pxPrevious = ( ListItem_t * ) &( pxList->xListEnd );

    /* 初始化时，列表中的列表项数量为 0（不包含末尾列表项 xListEnd） */
    pxList->uxNumberOfItems = ( UBaseType_t ) 0U;

    /* 初始化用于检测列表数据完整性的校验值 */
    listSET_LIST_INTEGRITY_CHECK_1_VALUE( pxList );
    listSET_LIST_INTEGRITY_CHECK_2_VALUE( pxList );
}
```

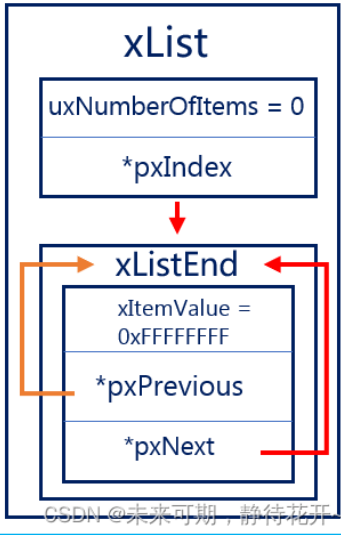
CSDN @未来可期，静

列表初始化函数vListInitialise()

- 1、初始化时，列表中只有 xListEnd，因此 pxIndex 指向 xListEnd
- 2、xListEnd 的值初始化为最大值，用于列表项升序排序时，排在最后
- 3、初始化时，列表中只有 xListEnd，因此上一个和下一个列表项都为 xListEnd
- 4、初始化时，列表中的列表项数量为 0（不包含 xListEnd）

CSDN @未来可期，静待

初始化后列表结构



CSDN @未来可期，静待花开

一开始，列表为空，只有一个末尾列表项，因此：

- 1. 用于记录列表中列表项的个数的变量uxNumberOfItems应为0；
- 2. 成员 pxIndex 用于指向列表中的某个列表项，此时应该指向末尾列表项；
- 3. 末尾列表项的成员变量 xItemValue 应该是最大的，因为要保证放在最后一个位置；
- 4. 末尾列表项的前驱结点指针和后继结点指针都应该指向它自己（和双向循环链表初始化一样）；

2.2 初始化列表项函数vListInitialiseItem()

cpp

AI写代码

```
1 函数原型
2 void vListInitialiseItem( ListItem_t * const pxItem );
```

形参	描述
pxItem	待初始化列表项

```
void vListInitialiseItem( ListItem_t * const pxItem )
{
    /* 初始化时，表示当前没有任何对象（如任务控制块）拥有该列表项*/
    pxItem->pxContainer = NULL;

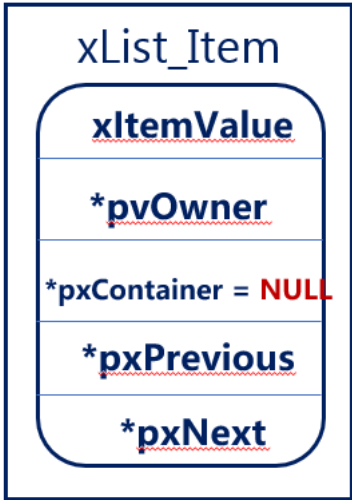
    /* 初始化用于检测列表项数据完整性的校验值 */
    listSET_FIRST_LIST_ITEM_INTEGRITY_CHECK_VALUE( pxItem );
    listSET_SECOND_LIST_ITEM_INTEGRITY_CHECK_VALUE( pxItem );
}
```

CSDN @未来可期，静

列表项初始化函数vListInitialiseltm()

1、初始化时，列表项不属于任何一个列表，所以为空

初始化后的列表项结构



CSDN @未来可期，静待花开~

2.3 列表项升序插入函数vListInsert()

2.3.1 源码分析

cpp

AI写代码

1 函数原型

2 void vListInsert (List_t * const pxList , ListItem_t * const pxNewListItem)

3

形参	描述
pxList	插入到哪个列表
pxNewListItem	待插入列表项

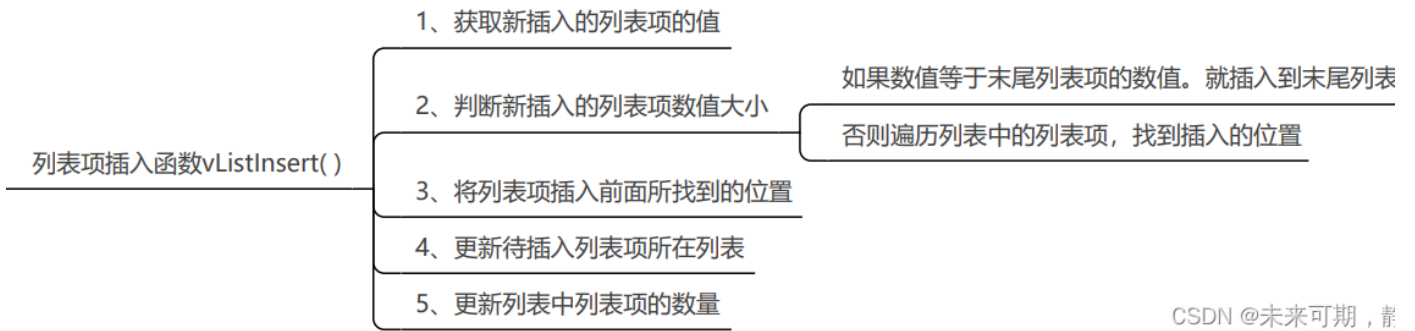
此函数用于将待插入列表的列表项按照列表项值升序进行排序，有序地插入到列表中!


```
void vListInsert( List_t * const pxList, ListItem_t * const pxNewListItem )
{
    ListItem_t * pxIterator;
    const TickType_t xValueOfInsertion = pxNewListItem->xItemValue; /* 获取列表项的数值依据数值升序排列 */
    listTEST_LIST_INTEGRITY( pxList ); /* 检查参数是否正确 */
    listTEST_LIST_ITEM_INTEGRITY( pxNewListItem );

    if( xValueOfInsertion == portMAX_DELAY ) /* 如果待插入列表项的值为最大值 */
    {
        pxIterator = pxList->xListEnd.pxDPrevious; /* 插入的位置为列表 xListEnd 前面 */
    }
    else
    {
        /* 找到合适的插入位置 */
        for( pxIterator = ( ListItem_t * ) &( pxList->xListEnd ); /* 遍历列表中的列表项，找到插入的位置 */
            { pxIterator->pxNext->xItemValue <= xValueOfInsertion;
              pxIterator = pxIterator->pxNext } )
        {
        }

        pxNewListItem->pxNext = pxIterator->pxNext;
        pxNewListItem->pxNext->pxPrevious = pxNewListItem;
        pxNewListItem->pxPrevious = pxIterator;
        pxIterator->pxNext = pxNewListItem;
        pxNewListItem->pxContainer = pxList;
        ( pxList->uxNumberOfItems )++;
    }
}
```

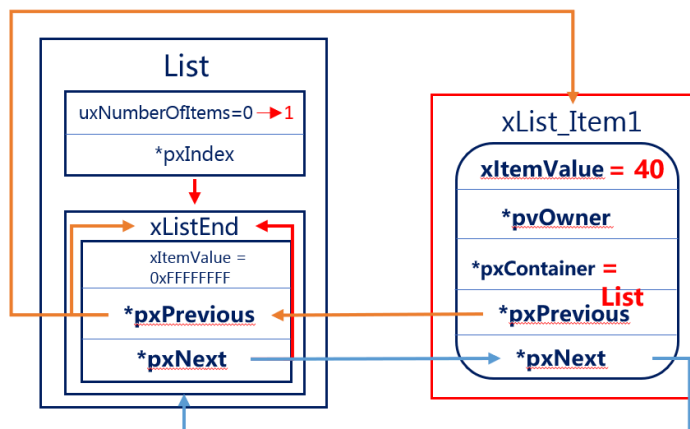
CSDN @未来可期



可以看到：函数vListInsert() 跟双向循环链表的按位置插入结点思想相同。

2.3.2 举例理解

插入值为40的列表项

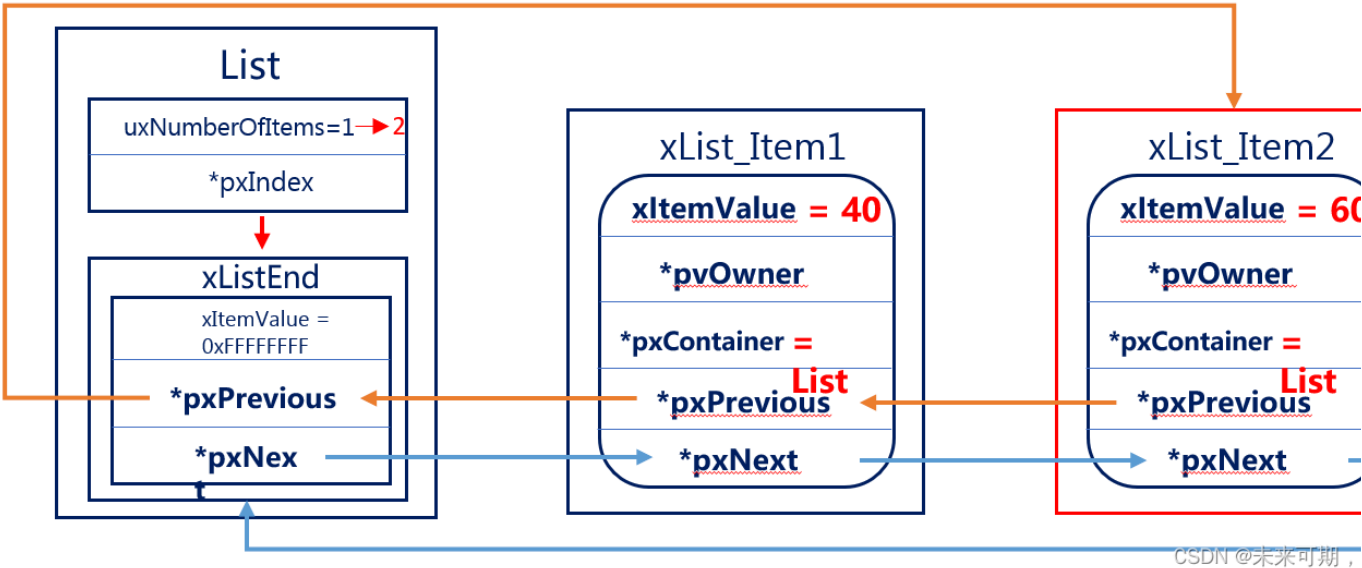


将列表项xList_Item1，插入列表List中

CSDN @未来可期，静待花开~

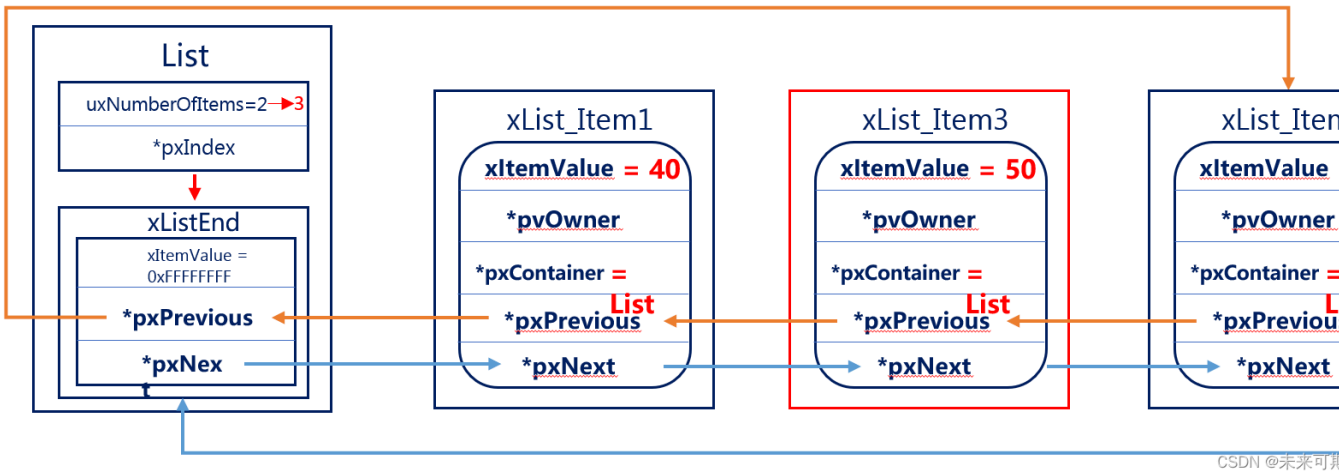
插入值为60的列表项

在前面的基础上，在插入值为60的列表项，结果如下：



插入值为50的列表项

在前面的基础上，在插入值为50的列表项，结果如下：



总结：函数vListInsert()，是将待插入列表的列表项按照列表项值升序进行排序，有序地插入到列表中

2.4 列表项尾插函数vListInsertEnd()

2.4.1 源码分析

cpp

```
1 函数原型
2 void vListInsertEnd( List_t * const pxList, ListItem_t * const pxNewListItem );
```

AI写代码

形参	描述
pxList	插入到哪个列表
pxNewListItem	待插入列表项

```

void vListInsertEnd ( List_t * const pxList , ListItem_t * const pxNewListItem
{
    省略部分非关键代码 ... ..
    /* 获取列表 pxIndex 指向的列表项 */
    ListItem_t * const pxIndex = pxList->pxIndex;

    /* 更新待插入列表项的指针成员变量 */
    pxNewListItem->pNext = pxIndex;
    pxNewListItem->pPrevious = pxIndex->pPrevious;

    /* 更新列表中原本列表项的指针成员变量 */
    pxIndex->pPrevious->pNext = pxNewListItem;
    pxIndex->pPrevious = pxNewListItem;

    /* 更新待插入列表项的所在列表成员变量 */
    pxNewListItem->pxContainer = pxList;

    /* 更新列表中列表项的数量 */
    ( pxList->uxNumberOfItems )++;
}

```

修改指针的指向

CSDN @未来可期，静

末尾列表项插入函数vListInsertEnd()

1、获取列表 pxIndex 指向的列表项

2、将待插入的列表项插入到 pxIndex所指向的列表项前面

3、更新待插入列表项的所在列表

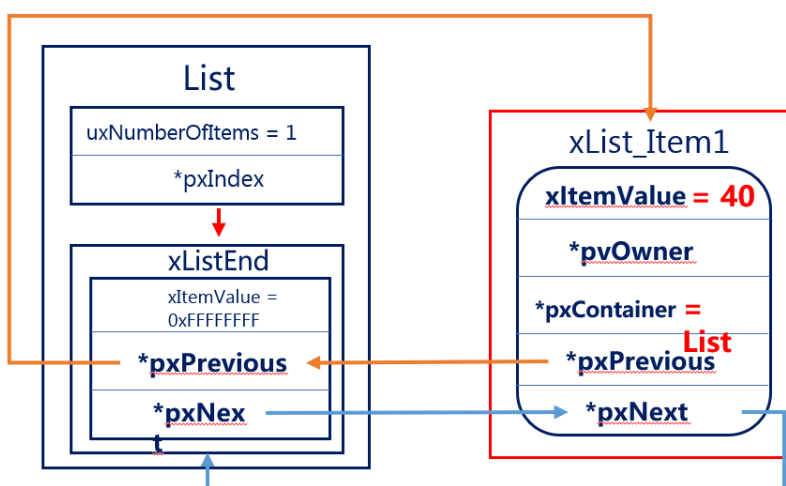
4、更新列表中列表项的数量

CSDN @未来可期，静待花开~

2.4.2 举例理解

列表项末尾插入图示

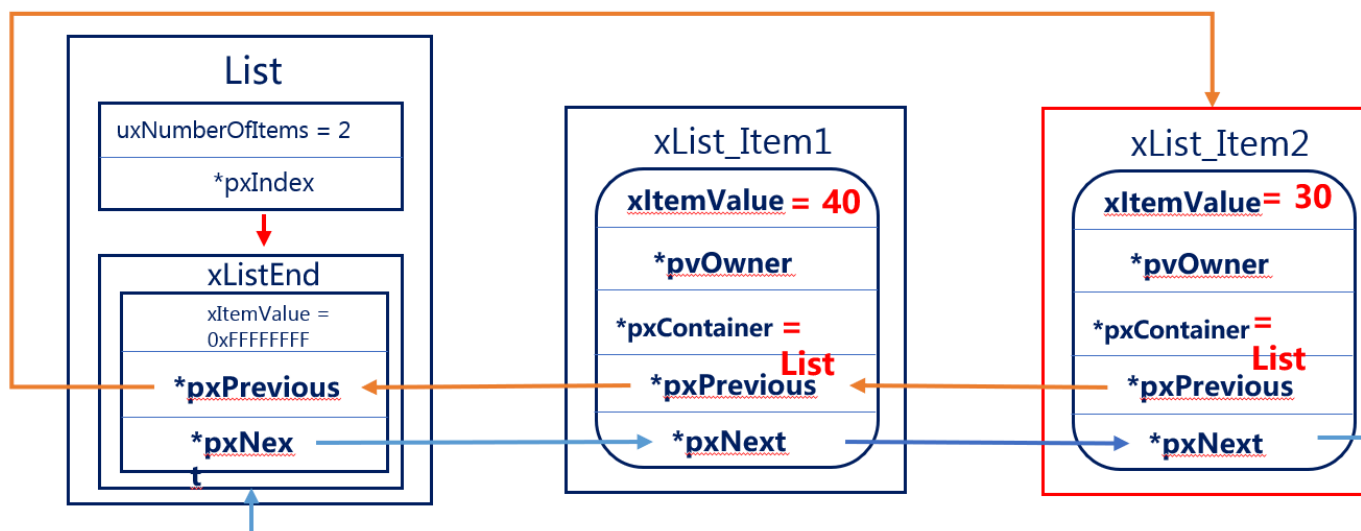
假设默认列表如下：



此时插入值为30的列表项2，结果

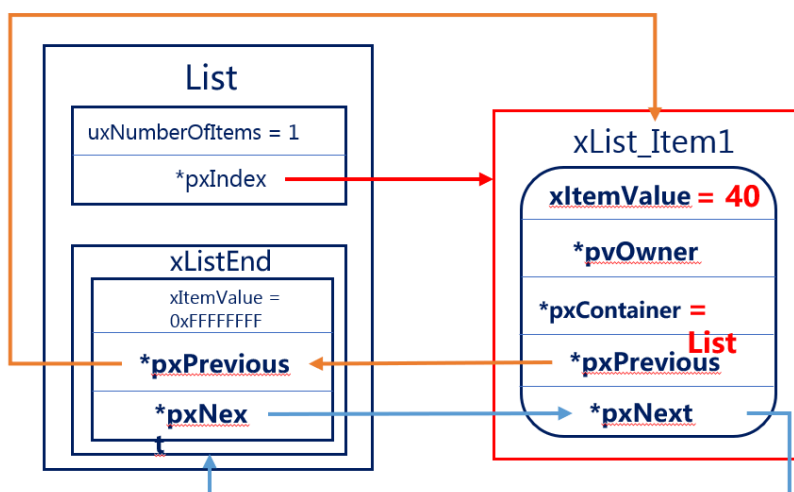
CSDN @未来可期

插入值为30的列表项



CSDN @未来可期，

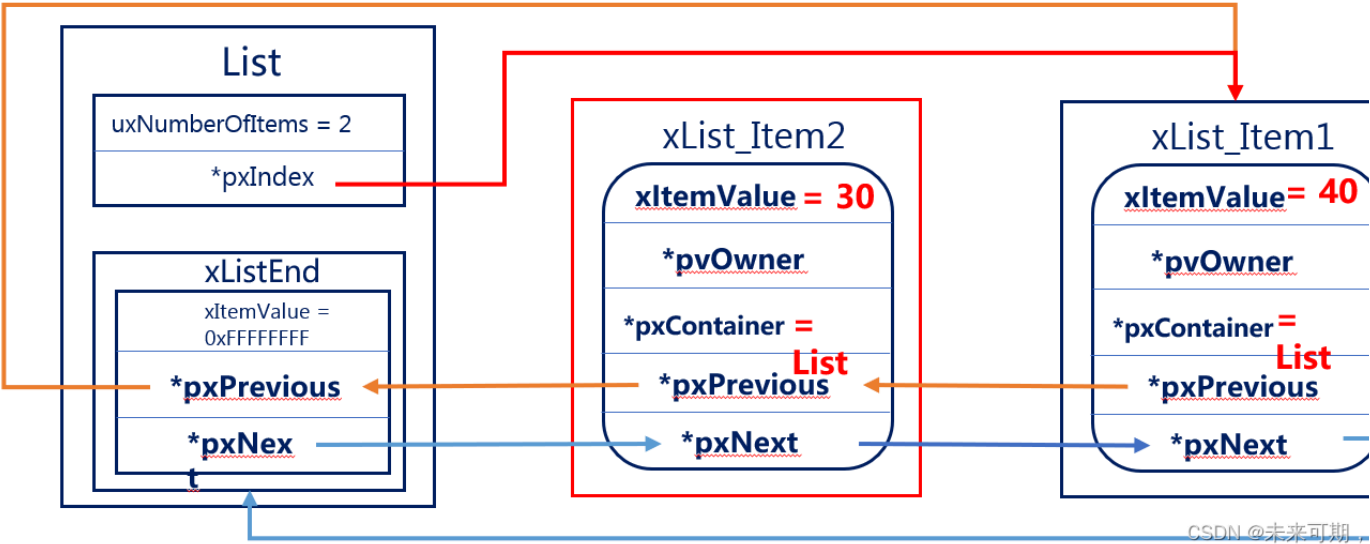
假设默认列表如下：



此时插入值为30的列表项2，结果

CSDN @未来可期

插入值为30的列表项



总结：函数vListInsertEnd(), 是将待插入的列表项插入到列表 pxIndex 指针指向的列表项前面;它是一种无序的插入方法！！

2.5 列表项删除函数uxListRemove()

cpp AI写代码

```
1 函数原型
2 UBaseType_t uxListRemove ( ListItem_t * const pxItemToRemove )
3
```

此函数用于将列表项从列表项所在列表中移除。

形参	描述
pxItemToRemove	待移除的列表项
返回值	描述
整数	待移除列表项移除后，所在列表剩余列表项的数量

```

UBaseType_t uxListRemove( ListItem_t * const pxItemToRemove )
{
    /* 获取待删除列表项所在的列表 */
    List_t * const pxList = pxItemToRemove->pxContainer;

    /* 从列表中移除列表项（跨越指针） */
    pxItemToRemove->pxNext->pxPrevious = pxItemToRemove->pxPrevious;
    pxItemToRemove->pxPrevious->pxNext = pxItemToRemove->pxNext;

    /* 如果 pxIndex 正指向待移除的列表项 */
    if( pxList->pxIndex == pxItemToRemove )
    {
        /* pxIndex 指向上一个列表项 */
        pxList->pxIndex = pxItemToRemove->pxPrevious;
    } else
    {
        mtCOVERAGE_TEST_MARKER();
    }

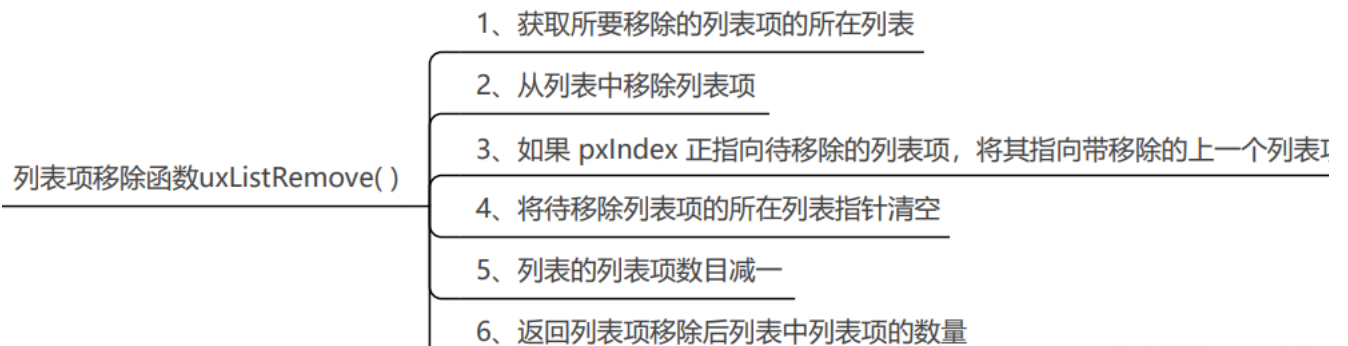
    /* 将待移除的列表项的所在列表指针清空 */
    pxItemToRemove->pxContainer = NULL;

    /* 更新列表中列表项的数量 */
    ( pxList->uxNumberOfItems )--;

    /* 返回移除后的列表中列表项的数量 */
    return pxList->uxNumberOfItems;
}

```

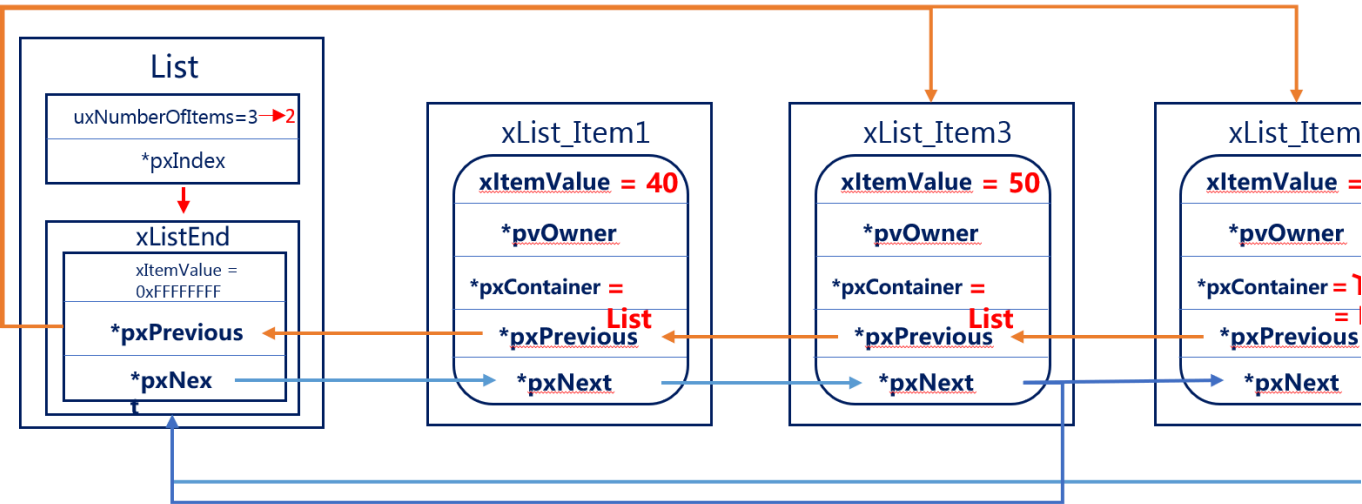
CSDN @未来可期



CSDN @未来可期，静待花开

列表项的删除图示

移除xList_Item2



CSDN @未来可期

三、列表项的插入和删除实验

- 1、实验目的：学会对FreeRTOS 列表和列表项的操作函数使用，并观察运行结果和析是否一致
- 2、实验设计：将设计三个任务：start_task、task1、task2

三个任务的功能如下：

- start_task 用来创建其他的2个任务
- task1 实现LED0每500ms闪烁一次，用来提示系统正在运行
- task2 调用列表和列表项相关API函数，并且通过串口输出相应的信息，进行观察

CSDN @未来可期

本实验本质上其实就是对双向链表进行增删操作，为了验证结果，打印列表项的地址和前驱结点地址以及后继结点地址进行验证即可。

3.1 创建任务及实现任务函数

cppAI写代码

```
1  #include "stm32f4xx.h"           // Device header
2  #include "stdio.h"
3  #include "FreeRTOS.h"
4  #include "task.h"
5  #include "dynamic.h"
6  #include "mydelay.h"
7
8
9
10 /*****START_TASK 任务配置*****/
11 /***** 包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
12
13 #define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
14 #define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
15 TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
16 void start_task(void* args);
17
```

```

18 | 19 |
19 |
20 |
21 | /*****TASK1任务配置*****/
22 | /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
23 | #define TASK1_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
24 | #define TASK1_PRIIO 2 //定义任务优先级，0-31根据任务需求
25 | TaskHandle_t task1_handler; //定义任务句柄（结构体指针）
26 | void task1(void* args);
27 |
28 |
29 |
30 |
31 | /*****TASK2任务配置*****/
32 | /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
33 | #define TASK2_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
34 | #define TASK2_PRIIO 3 //定义任务优先级，0-31根据任务需求
35 | TaskHandle_t task2_handler; //定义任务句柄（结构体指针）
36 | void task2(void* args);
37 |
38 |
39 |
40 | /*****开始任务用来创建一个任务，只创建一次，不能是死循环,创建完这个任务后删除开始任务本身*****/
41 | void start_task(void* args)
42 | {
43 |     taskENTER_CRITICAL(); /*进入临界区*/
44 |
45 |     xTaskCreate( (TaskFunction_t) task1,
46 |                 (char *) "task1",
47 |                 ( configSTACK_DEPTH_TYPE) TASK1_STACK_SIZE,
48 |                 (void *) NULL,
49 |                 (UBaseType_t) TASK1_PRIIO ,
50 |                 (TaskHandle_t *) &task1_handler );
51 |
52 |
53 |     xTaskCreate( (TaskFunction_t) task2,
54 |                 (char *) "task2",
55 |                 ( configSTACK_DEPTH_TYPE) TASK2_STACK_SIZE,
56 |                 (void *) NULL,
57 |                 (UBaseType_t) TASK2_PRIIO ,
58 |                 (TaskHandle_t *) &task2_handler );
59 |
60 |
61 |
62 |     vTaskDelete(NULL); //删除开始任务自身，传参NULL
63 |
64 |     taskEXIT_CRITICAL(); /*退出临界区*/
65 |
66 |
67 |     //临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
68 | }
69 |
70 |
71 |
72 |
73 | /*****任务1的任务函数，无返回值且是死循环*****/
74 |
75 | /***任务1：实现LED0每500ms翻转一次*****/
76 | void task1(void* args)
77 | {
78 |
79 |     while(1)
80 |     {
81 |         printf("22222\n");
82 |         GPIO_ToggleBits(GPIOF,GPIO_Pin_9 );
83 |         vTaskDelay(500); //FreeRTOS自带的延时函数,延时500毫秒
84 |
85 |     }
86 | }
87 |
88 |
89 | List_t TestList; /*定义测试列表*/

```



```

90 | ListItem_t   ListItem1;    /*定义测试列表项1*/
91 | ListItem_t   ListItem2;    /*定义测试列表项2*/
92 | ListItem_t   ListItem3;    /*定义测试列表项3*/
93
94
95
96 | /***** 任务2的任务函数，无返回值且是死循环*****/
97
98 | /** 任务2：列表项插入和删除实验*****/
99 void task2(void* args)
100 {
101     vListInitialise( &TestList );          /*初始化列表*/
102     vListInitialiseItem( &ListItem1);       /*初始化列表项1*/
103     vListInitialiseItem( &ListItem2);       /*初始化列表项2*/
104     vListInitialiseItem( &ListItem3);       /*初始化列表项3*/
105     ListItem1.xItemValue=40;
106     ListItem2.xItemValue=60;
107     ListItem3.xItemValue=50;
108
109     /*第二步:打印列表和其他列表项的地址*/
110     printf("/*****第二步:打印列表和列表项的地址*****/\r\n");
111     printf("项目\t\t\t地址\r\n" );
112     printf("TestList\t\t0x%p\r\n",&TestList);          // 列表地址
113     printf("TestList->pxIndex\t0x%p\r\n",TestList.pxIndex);    // 列表中的列表项指针指向的列表项的地址
114     printf("TestList->xListEnd\t0x%p\r\n",&TestList.xListEnd)); // 末尾列表项的地址
115     printf("ListItem1\t\t0x%p\r\n",&ListItem1);          // 列表项1的地址
116     printf("ListItem2\t\t0x%p\r\n",&ListItem2);          // 列表项2的地址
117     printf("ListItem3\t\t0x%p\r\n",&ListItem3);          // 列表项3的地址
118     printf (" /*****结束*****/\r\n");
119
120
121     /*第三步:列表项1插入列表: */
122     printf("\r\n*****第三步:列表项1插入列表*****/\r\n");
123     vListInsert((List_t* )&TestList,          /* 列表 */
124                 (ListItem_t*) &ListItem1);    /* 列表项 */
125     printf("项目\t\t\t\t地址:\r\n");
126     printf("TestList->xListEnd->pxNext\t0x%p\r\n ",(TestList.xListEnd.pxNext));
127     printf("ListItem1->pxNext\t\t0x%p\r\n", (ListItem1.pxNext));
128     printf("TestList->xListEnd->pxPrevious\t0x%p\r\n", (TestList.xListEnd.pxPrevious));
129     printf("ListItem1->pxPrevious\t\t0x%p\r\n", (ListItem1.pxPrevious));
130     printf (" /*****结束*****/\r\n");
131
132
133     /*第四步:列表项2插入列表: */
134     printf ("\r\n /*****第四步:列表项2插入列表*****/\r\n");
135     vListInsert((List_t* )&TestList,          /* 列表 */
136                 (ListItem_t*)&ListItem2);    /* 列表项 */
137     printf("项目\t\t\t\t\t地址\r\n" );
138     printf("TestList->xListEnd->pxNext\t0x%p\r\n", (TestList.xListEnd.pxNext));
139     printf("ListItem1->pxNext\t\t0x%p\r\n", (ListItem1.pxNext));
140     printf("ListItem2->pxNext\t\t0x%p\r\n", (ListItem2.pxNext));
141     printf("TestList->xListEnd->pxPrevious\t0x%p\r\n", (TestList.xListEnd.pxPrevious));
142     printf("ListItem1->pxPrevious\t\t0x%p\r\n", (ListItem1. pxPrevious));
143     printf("ListItem2->pxPrevious\t\t0x%p\r\n", (ListItem2. pxPrevious));
144     printf (" /*****结束*****/\r\n");
145
146
147     /* 第五步:列表项3插入列表 */
148     printf("\r\n*****第五步:列表项3插入列表*****/\r\n");
149     vListInsert((List_t* ) &TestList,          /* 列表 */
150                 (ListItem_t*)&ListItem3);    /* 列表项 */
151     printf("项目\t\t\t\t\t地址\r\n");
152     printf("TestList->xListEnd->px.Next\t0x%p\r\n", (TestList.xListEnd.pxNext));
153     printf("ListItem1->pxNext\t\t0x%p\r\n", (ListItem1.pxNext));
154     printf("ListItem2->pxNext\t\t0x%p\r\n", (ListItem2.pxNext));
155     printf("ListItem3->pxNext\t\t0x%p\r\n", (ListItem3.pxNext));
156     printf("TestList->xListEnd->pxPrevious\t0x%p\r\n", (TestList.xListEnd.pxPrevious));
157     printf("ListItem1->pxPrevious\t\t0x%p\r\n", (ListItem1.pxPrevious));
158     printf("ListItem2->pxPrevious\t\t0x%p\r\n", (ListItem2.pxPrevious));
159     printf("ListItem3->pxPrevious\t\t0x%p\r\n", (ListItem3.pxPrevious));
160     printf("/*****结束*****/\r\n");

```

```

161 |
162 |
163 | /* 第六步: 移除列表项2 */
164 | printf("\r\n *****第六步:移除列表项2*****\r\n");
165 | uxListRemove((ListItem_t*)&ListItem2); /* 移除列表项 */
166 | printf("项目\t\t\t\t地址\r\n");
167 | printf("TestList->xListEnd->pxNext\t\t0x%p\r\n", (TestList.xListEnd.pxNext));
168 | printf("ListItem1->pxNext\t\t0x%p\r\n", (ListItem1.pxNext));
169 | printf("ListItem3->pxNext\t\t0x%p\r\n", (ListItem3.pxNext));
170 | printf("TestList->xListEnd->pxPrevious\t\t0x%p\r\n", (TestList.xListEnd.pxPrevious));
171 | printf("ListItem1->pxPrevious\t\t0x%p\r\n", (ListItem1.pxPrevious));
172 | printf("ListItem3->pxPrevious\t\t0x%p\r\n", (ListItem3.pxPrevious));
173 | printf("/*****结束*****\r\n");
174 |
175 | /* 第七步: 列表末尾添加列表项2 */
176 | printf("\r\n/ *****第七步:列表末尾添加列表项2*****\r\n");
177 | vListInsertEnd((List_t*)&TestList, /* 列表 */
178 |               (ListItem_t*)&ListItem2); /* 列表项 */
179 | printf("项目\t\t\t\t地址\r\n");
180 | printf("TestList->pxIndex\t\t0x%p\r\n", TestList.pxIndex);
181 | printf("TestList->xListEnd->pxNext\t\t0x%p\r\n", (TestList.xListEnd.pxNext));
182 | printf("ListItem1->pxNext\t\t0x%p\r\n", (ListItem1.pxNext));
183 | printf("ListItem2->pxNext\t\t0x%p\r\n", (ListItem2.pxNext));
184 | printf("ListItem3->pxNext\t\t0x%p\r\n", (ListItem3.pxNext));
185 | printf("TestList->xListEnd->pxPrevious\t\t0x%p\r\n", (TestList.xListEnd.pxPrevious));
186 | printf("ListItem1->pxPrevious\t\t0x%p\r\n", (ListItem1.pxPrevious));
187 | printf("ListItem2->pxPrevious\t\t0x%p\r\n", (ListItem2.pxPrevious));
188 | printf("ListItem3->pxPrevious\t\t0x%p\r\n", (ListItem3.pxPrevious));
189 | printf("/*****实验结束*****\r\n");
190 |
191 | while(1)
192 | {
193 |     printf("11111\n");
194 |     vTaskDelay(1000); //FreeRTOS自带的延时函数, 延时1秒
195 | }
196 | }
197 |
198 |
199 |
200 |
201 |
202 | //FreeRTOS入口例程函数, 无参数, 无返回值
203 | void freertos_demo(void)
204 | {
205 |     xTaskCreate( (TaskFunction_t) start_task,
206 |                 (char *) "start_task",
207 |                 ( configSTACK_DEPTH_TYPE) START_TASK_STACK_SIZE,
208 |                 (void *) NULL,
209 |                 (UBaseType_t) START_TASK_PRIOR ,
210 |                 (TaskHandle_t *) &start_task_handler );
211 |
212 |
213 |     vTaskStartScheduler(); // 开启任务调度器
214 |
215 | }
216 |
217 |

```

3.2 主函数调用 入口函数, 操作系统 开始进行任务的切换和调度

cpp

AI写代码

```

1 | #include "stm32f4xx.h" // Device header
2 | #include "stdio.h"
3 | #include "myled.h"
4 | #include "myusart.h"
5 |
6 |
7 | #include "FreeRTOS.h"
8 | #include "task.h"
9 | #include "dynamic.h"

```

```

10 | 11 | extern TaskHandle_t Start_Handle;
12 |
13 | int main(void)
14 | {
15 |     // 硬件初始化
16 |     My_UsartInit();
17 |
18 |
19 |     // 调用入口函数
20 |     freertos_demo();
21 |
22 | }
23 |

```

3.3 实验结果

```

/*****第二步：打印列表和列表项的地址*****/
项目      地址
TestList      0x200000d4
TestList->pxIndex      0x200000dc
TestList->pxListEnd      0x200000dc
ListItem1      0x200000e8
ListItem2      0x200000fc
ListItem3      0x20000110
/*****结束*****/

```

```

/*****第三步：列表项1插入列表*****/
项目      地址
TestList->pxListEnd->pxNext      0x200000e8
ListItem1->pxNext      0x200000dc
TestList->pxListEnd->pxPrevious      0x200000e8
ListItem1->pxPrevious      0x200000dc
/*****结束*****/

```

```

/*****第四步：列表项2插入列表*****/
项目      地址
TestList->pxListEnd->pxNext      0x200000e8
ListItem1->pxNext      0x200000fc
ListItem2->pxNext      0x200000dc
TestList->pxListEnd->pxPrevious      0x200000fc
ListItem1->pxPrevious      0x200000dc
ListItem2->pxPrevious      0x200000e8
/*****结束*****/

```

```

/*****第五步：列表项3插入列表*****/
项目      地址
TestList->pxListEnd->pxNext      0x200000e8
ListItem1->pxNext      0x20000110
ListItem2->pxNext      0x200000dc
ListItem3->pxNext      0x200000fc
TestList->pxListEnd->pxPrevious      0x200000fc
ListItem1->pxPrevious      0x200000dc
ListItem2->pxPrevious      0x20000110
ListItem3->pxPrevious      0x200000e8
/*****结束*****/

```

```

/*****第六步：移除列表项2*****/
项目      地址
TestList->pxListEnd->pxNext      0x200000e8
ListItem1->pxNext      0x20000110
ListItem3->pxNext      0x200000dc
TestList->pxListEnd->pxPrevious      0x20000110
ListItem1->pxPrevious      0x200000dc
ListItem3->pxPrevious      0x200000e8
/*****结束*****/

```

```

/*****第七步：列表末尾添加列表项2*****/
项目      地址
TestList->pxIndex      0x200000dc
TestList->pxListEnd->pxNext      0x200000e8
ListItem1->pxNext      0x20000110
ListItem2->pxNext      0x200000dc
ListItem3->pxNext      0x200000fc
TestList->pxListEnd->pxPrevious      0x200000fc
ListItem1->pxPrevious      0x200000dc
ListItem2->pxPrevious      0x20000110
ListItem3->pxPrevious      0x200000e8
/*****实验结束*****/

```

CSDN @未来可期

根据每一步操作的地址，符合理论！

至此，已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加关注评论，您的支持是动力！下期再见！

标签 #FreeRTOS